codeHive

Stacks

LIFO: LAST IN, FIRST OUT

Part of the "Linear Structures" Series

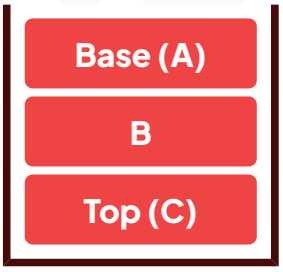
1. The Pancake Principle

A **Stack** is a linear data structure that follows the **LIFO (Last In First Out)** principle. Think of a stack of pancakes: the last pancake placed on top is the first one you eat.

LAST IN → FIRST OUT

Key Operations

- **Push:** Adds an element to the top.
- **Pop:** Removes the top element.
- **Peek:** Views the top element without removing it.
- **isEmpty:** Checks if the stack has elements.



2. Array Implementation

Implementing a stack with an array is memory-efficient and easy to understand. In Python, we use the `list` class.

```
class Stack:
    def __init__(self):
        self.items = []

    def push(self, element):
        self.items.append(element)

    def pop(self):
        if not self.isEmpty():
            return self.items.pop()
        return "Stack is empty"
```

```
    def peek(self):
        if not self.isEmpty():
            return self.items[-1]

    def isEmpty(self):
        return len(self.items) == 0

# usage
myStack = Stack()
myStack.push('A')
myStack.push('B')
print(myStack.peek()) # Returns 'B'
```

Pros: Fast access and low memory overhead per element.
Cons: Fixed size (in many languages) or occasional resizing overhead.

3. Linked List Implementation

Using a Linked List allows the stack to grow and shrink dynamically without memory reallocation concerns.

```
class Node:
    def __init__(self, value):
        self.value = value
        self.next = None

class Stack:
    def __init__(self):
        self.head = None

    def push(self, value):
        new_node = Node(value)
        # Point the new node to the old head
        new_node.next = self.head
        # New node becomes the top
        self.head = new_node

    def pop(self):
        if self.head is None: return "Empty"
        poppedValue = self.head.value
        self.head = self.head.next
        return poppedValue
```

Pros: Maximum flexibility; easy to handle vary large datasets.
Cons: Every value requires an extra "pointer" byte, increasing memory use.

4. Real-World Applications

Undo/Redo Mechanisms

Every time you press `Ctrl+Z`, the software "pops" your last action off a command stack to revert state.

Backtracking Algorithms

Used in AI for maze solving or game trees where the program needs to "go back" to a previous node in a graph.

5. Knowledge Check

Scenario: You push "Red", then "Green", then "Blue". You call `pop()` then `peek()`. What is the final result of the peek?

Answer: "Green"